
SOMMAIRE

Introduction	1
Présentation du jeu de donnéesAnalyse 3.	2
État de l'art	3
Preprocessing des données	4
Analyse Exploratoire des Données	5
Analyse Exploratoire des Données (EDA)	6
Entraînement et validation des modèles	7
Visualisation des résultats	8
Power BI	9
Interface	10

1. Introduction

1.1 Contexte du projet

La gestion efficace de l'énergie est devenue un enjeu majeur pour les collectivités territoriales et les bâtiments publics. Face à l'augmentation des coûts de l'électricité et aux objectifs de réduction des émissions de gaz à effet de serre, l'optimisation de la consommation énergétique constitue une priorité.

Les hôtels de ville, en tant que bâtiments publics à forte fréquentation, présentent des profils de consommation électrique complexes, influencés par de nombreux facteurs : occupation humaine, conditions météorologiques, jours fériés, etc. La capacité à comprendre et à prévoir cette consommation permet de :

- Réduire les factures énergétiques
- Mieux dimensionner les contrats d'approvisionnement
- Détecter les anomalies de consommation (gaspillage, dysfonctionnements)
- Planifier des actions d'efficacité énergétique

1.2 Problématique

Ce projet s'inscrit dans cette dynamique et vise à répondre à la question suivante :
Comment analyser et prédire la consommation électrique d'un hôtel de ville à partir de données historiques de mesure à pas de temps fin (15 minutes) ?

Plus précisément, il s'agit de :

- Caractériser les profils de consommation (cycles journaliers, saisonnalité)
- Identifier les anomalies et les périodes de données manquantes
- Prédire la consommation future à court et moyen terme

2. Présentation du jeu de données

2.1 Description générale

Le jeu de données utilisé dans ce projet, intitulé city-hall-electricity-use.csv, contient des mesures de consommation électrique d'un bâtiment public,

Caractéristique	Description
Type de données	Série temporelle univariée
Période couverte	Janvier 2016 - Mars 2020 (environ 26 mois)
Granularité temporelle	Pas de temps de 15 minutes
Fréquence d'échantillonnage	96 mesures par jour

2.2 Structure des données

Le fichier se compose de deux colonnes distinctes :

Colonne	Type	Description
DateTime_Measured	Datetime	Date et heure de la mesure (format YYYY-MM-DD HH:MM:SS)
Total_Demand_KW	Numérique	Demande électrique en kilowatts (kW)

Les données sont organisées de manière chronologique décroissante (du plus récent au plus ancien).

3. État de l'art

La gestion de l'énergie dans les bâtiments publics est un enjeu majeur d'efficacité énergétique et de réduction des coûts. L'analyse des données de consommation électrique à haute fréquence (pas de temps de 15 minutes) permet de :

- Comprendre les profils de consommation et les pics de demande.
- Détecter des anomalies (e.g., consommation excessive, oubli d'extinction de systèmes).
- Prévoir la consommation future pour optimiser l'approvisionnement ou détecter des dérives

1. Nature du problème

Le problème principal est un **problème de prédiction de séries temporelles multivariées**.

Plus précisément, il s'agit d'une **régression** où la variable cible est continue (la demande en kW), et l'objectif est de prédire une séquence future en se basant sur l'historique.

Deux sous-problèmes peuvent être définis :

1. **Prédiction à court terme (intrajournalière)** : Prédire la consommation des prochaines heures (H+1 à H+6). Utile pour le pilotage en temps réel.
2. **Prédiction à moyen terme** : Prédire la consommation du lendemain. Utile pour la gestion contractuelle (tarifs heures pleines/creuses).

Variables explicatives candidates :

- lag_1, lag_2, ... lag_96 (valeurs aux 96 pas de temps précédents)
- hour_of_day (0-23)
- day_of_week (0-6)
- month
- is_weekend (booléen)
- is_holiday (booléen, à ajouter avec des données externes)

3. Algorithmes sélectionnés

Modèle	Définition	Hyperparamètres clés	Complexité	Interprétabilité	Performances attendues	Temps d'entraînement
Régression linéaire	Modèle baseline qui	Aucun	$O(n \times d)$	Très haute	MAPE > 10%	< 1 sec
RandomForest	Ensemble d'arbres de	n_estimators, max_depth,	$O(n \times d \times k)$	Moyenne (feature)	MAPE 3-7%	1-3 min
XGBoost	Gradient boosting	n_estimators, learning_rate,	$O(n \times d \times k)$	Moyenne (feature)	MAPE 2-5%	1-5 min
LightGBM (défaut)	Gradient boosting leaf-	n_estimators, num_leaves,	$O(n \times d \times k)$	Moyenne (feature)	MAPE 2-5%	< 1 min
LightGBM (optimisé)	LightGBM avec hyperparamèt	n_estimators, max_depth,	$O(n \times d \times k)$	Moyenne (feature)	MAPE < 3%	2-15 min
LSTM	Réseau de neurones	epochs, batch_size,	$O(n \times T \times h^2)$	Faible (boîte noire)	MAPE 3-6%	5-20 min

4.Chargement des données

On a chargé le fichier de données city-hall-electricity-use.csv dans Google Colab en utilisant la méthode d'upload direct.

```
# --- Option A : upload direct dans Colab ---
from google.colab import files
uploaded = files.upload() # sélectionnez city-hall-electricity-use.csv

# --- Option B : depuis Google Drive (décommentez) ---
# from google.colab import drive
# drive.mount("/content/drive")
# DATA_PATH = "/content/drive/MyDrive/city-hall-electricity-use.csv"

DATA_PATH = "city-hall-electricity-use.csv"
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving city-hall-electricity-use.csv to city-hall-electricity-use.csv

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

df_raw = pd.read_csv(DATA_PATH, parse_dates=["DateTime_Measured"])
print(f"Dimensions : {df_raw.shape}")
print(f"Période : {df_raw['DateTime_Measured'].min()} → {df_raw['DateTime_Measured'].max()}")
df_raw.head()
```

Dimensions : (106766, 2)
Période : 2016-10-27 09:30:00 → 2020-03-26 15:00:00

	DateTime_Measured	Total_Demand_KW
0	2020-03-26 15:00:00	1313
1	2020-03-26 14:45:00	1317
2	2020-03-26 14:30:00	1328
3	2020-03-26 14:15:00	1324
4	2020-03-26 14:00:00	1324

On a importé les bibliothèques nécessaires (Pandas, NumPy, Matplotlib, Seaborn) puis on a vérifié :

- Les dimensions du jeu de données : 106 766 lignes et 2 colonnes
- La période couverte : du 27 octobre 2016 au 26 mars 2020

Les 5 premières lignes du tableau, montrant les colonnes DateTime_Measured (date/heure) et Total_Demand_KW (consommation électrique en kW).

5.Preprocessing des données

1 Nettoyage

1.1 Suppression des doublons

```
# — Suppression des doublons —————
nb_doublons = df_raw.duplicated(subset=["DateTime_Measured"]).sum()
print(f"Doublons détectés : {nb_doublons}")

df = df_raw.copy()
df = df.groupby("DateTime_Measured", as_index=False)["Total_Demand_KW"].mean()
print(f"Lignes après suppression doublons : {len(df)}")
```

```
Doublons détectés : 48
Lignes après suppression doublons : 106718
```

On a supprimé les doublons dans la colonne DateTime_Measured.

- 48 doublons ont été détectés.
- On a regroupé les données par date/heure en prenant la moyenne des valeurs Total_Demand_KW pour les doublons.

Après suppression, le jeu de données contient désormais 106 718 lignes.

1.2 Traitement des valeurs manquantes et incohérences

```
# — Valeurs nulles et valeurs <= 0 —————
print("Valeurs nulles :", df["Total_Demand_KW"].isna().sum())
print("Valeurs <= 0 :", (df["Total_Demand_KW"] <= 0).sum())

# Suppression des valeurs incohérentes (<= 0)
df = df[df["Total_Demand_KW"] > 0].copy()

# Réindexation sur une grille 15 min complète
df = df.sort_values("DateTime_Measured").set_index("DateTime_Measured")
full_range = pd.date_range(start=df.index.min(), end=df.index.max(), freq="15min")
df = df.reindex(full_range)

missing_before = df["Total_Demand_KW"].isna().sum()
df["Total_Demand_KW"] = df["Total_Demand_KW"].interpolate(method="linear")
missing_after = df["Total_Demand_KW"].isna().sum()
print(f"Valeurs manquantes : {missing_before} → {missing_after} (après interpolation linéaire)")
```

```
Valeurs nulles : 0
Valeurs <= 0 : 7341
Valeurs manquantes : 20291 → 0 (après interpolation linéaire)
```

On a vérifié et traité les valeurs incohérentes :

- Aucune valeur nulle n'a été détectée.
- 7 341 valeurs ≤ 0 (incohérentes pour une consommation électrique) ont été supprimées.
- On a réindexé les données sur une grille horaire complète de 15 minutes entre la première et la dernière date.
- 20 291 valeurs manquantes ont été détectées après réindexation.
- On a appliqué une interpolation linéaire pour combler ces valeurs manquantes.
- Après interpolation, il ne reste plus aucune valeur manquante.

5. Preprocessing des données

2 Encodage des variables catégorielles

```
# Les variables temporelles sont déjà numériques (entiers).
# On crée les features de base et on applique un Label Encoding pour is_weekend.

df["hour"] = df.index.hour
df["dayofweek"] = df.index.dayofweek
df["month"] = df.index.month
df["year"] = df.index.year
df["quarter"] = df.index.quarter
df["is_weekend"] = (df.index.dayofweek >= 5).astype(int) # Label Encoding binaire

print("Variables créées :", ["hour", "dayofweek", "month", "year", "quarter", "is_weekend"])
df.head()
```

Variables créées : ['hour', 'dayofweek', 'month', 'year', 'quarter', 'is_weekend']

	Total_Demand_KW	hour	dayofweek	month	year	quarter	is_weekend
2016-10-27 09:30:00	1716.0	9	3	10	2016	4	0
2016-10-27 09:45:00	1724.0	9	3	10	2016	4	0
2016-10-27 10:00:00	1731.0	10	3	10	2016	4	0
2016-10-27 10:15:00	1724.0	10	3	10	2016	4	0
2016-10-27 10:30:00	1739.0	10	3	10	2016	4	0

On a créé des variables temporelles à partir de l'index datetime pour enrichir les données puis
On a visualisé les 5 premières lignes du tableau avec les nouvelles colonnes ajoutées.

3 Feature Engineering

```
# Lags temporels : 15 min (lag 1), 1h (lag 4), 1 jour (lag 96)
for lag in [1, 4, 96]:
    df[f"lag_{lag}"] = df["Total_Demand_KW"].shift(lag)

df = df.dropna()
print(f"Lignes après feature engineering : {len(df)}")
df.head()
```

Lignes après feature engineering : 119543

	Total_Demand_KW	hour	dayofweek	month	year	quarter	is_weekend	lag_1	lag_4	lag_96
2016-10-28 09:30:00	1635.0	9	4	10	2016	4	0	1059.0	1482.0	1716.0
2016-10-28 09:45:00	879.0	9	4	10	2016	4	0	1635.0	1509.0	1724.0
2016-10-28 10:00:00	1136.0	10	4	10	2016	4	0	879.0	1190.0	1731.0
2016-10-28 10:15:00	1651.0	10	4	10	2016	4	0	1136.0	1059.0	1724.0
2016-10-28 10:30:00	1639.0	10	4	10	2016	4	0	1651.0	1635.0	1739.0

On a créé des variables de décalage temporel (lags) pour capturer la dépendance avec les valeurs passées :

- lag_1 : décalage de 15 minutes (1 pas)
- lag_4 : décalage de 1 heure (4 pas de 15 min)
- lag_96 : décalage de 1 jour (96 pas de 15 min)

On a supprimé les lignes contenant des valeurs NaN (liées aux décalages).

On a visualisé les 5 premières lignes avec les nouvelles colonnes lag_1, lag_4 et lag_96 ajoutées.

5. Preprocessing des données

4 Détection et traitement des outliers

```
# — Détection via IQR —
Q1 = df["Total_Demand_KW"].quantile(0.25)
Q3 = df["Total_Demand_KW"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[(df["Total_Demand_KW"] < lower) | (df["Total_Demand_KW"] > upper)]
print(f"Seuil bas : {lower:.2f} kW")
print(f"Seuil haut : {upper:.2f} kW")
print(f"Outliers détectés : {len(outliers)} ({100*len(outliers)/len(df):.2f}%)")

# Boxplot avant traitement
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].boxplot(df["Total_Demand_KW"], vert=True)
axes[0].set_title("Boxplot avant traitement des outliers")
axes[0].set_ylabel("kW")

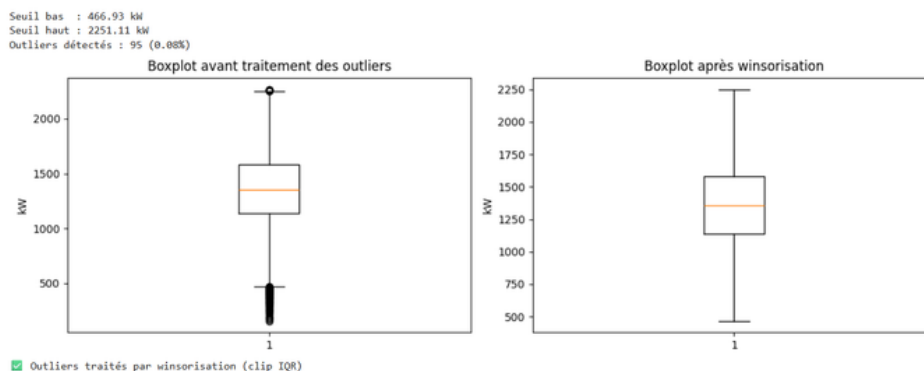
# Traitement : winsorisation (cap à IQR) plutôt que suppression pour conserver la série
df["Total_Demand_KW"] = df["Total_Demand_KW"].clip(lower=lower, upper=upper)

axes[1].boxplot(df["Total_Demand_KW"], vert=True)
axes[1].set_title("Boxplot après winsorisation")
axes[1].set_ylabel("kW")
plt.tight_layout()
plt.show()
print("✅ Outliers traités par winsorisation (clip IQR)")
```

On a détecté les valeurs aberrantes (outliers) sur la colonne Total_Demand_KW en utilisant la méthode de l'écart interquartile (IQR) :

- Seuil bas : 466,93 kW
- Seuil haut : 2 251,11 kW
- 95 outliers ont été détectés, soit 0,08% des données.

On a traité les outliers par winsorisation (plafonnement aux seuils IQR) plutôt que par suppression, afin de conserver la continuité de la série temporelle.



On a visualisé les boxplots avant et après traitement :

- Avant : présence de points aberrants au-delà des moustaches.
- Après : distribution resserrée sans valeurs extrêmes, les outliers ont été ramenés dans les limites des seuils.

5. Preprocessing des données

5 Normalisation / Standardisation

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

TARGET = "Total_Demand_KW"
features = [c for c in df.columns if c != TARGET]

# On utilise StandardScaler sur les features (pas sur la cible)
scaler = StandardScaler()
df_scaled = df.copy()
df_scaled[features] = scaler.fit_transform(df[features])

print("StandardScaler appliqué sur les features :", features)
print("\nMoyenne après scaling :", df_scaled[features].mean().round(4).values)
print("Écart-type après scaling :", df_scaled[features].std().round(4).values)

StandardScaler appliqué sur les features : ['hour', 'dayofweek', 'month', 'year', 'quarter', 'is_weekend', 'lag_1', 'lag_4', 'lag_96']

Moyenne après scaling : [ 0. -0.  0. -0. -0. -0.  0. -0.  0.]
Écart-type après scaling : [1.  1.  1.  1.  1.  1.  1.  1.  1.]
```

- On a appliqué un StandardScaler sur les variables features (toutes les colonnes sauf la cible Total_Demand_KW) pour les centrer et les réduire :
- Chaque feature a une moyenne de 0 et un écart-type de 1 après transformation.
- On a vérifié les résultats :
- Moyennes après scaling : toutes proches de 0 (avec de légères arrondis)
- Écarts-types après scaling : tous égaux à 1

6 Split des données : Train / Validation / Test

```
N = len(df_scaled)
train = df_scaled.iloc[:int(0.70 * N)]
val = df_scaled.iloc[int(0.70 * N):int(0.85 * N)]
test = df_scaled.iloc[int(0.85 * N):]

print(f"Train : {len(train)} lignes ({100*len(train)/N:.1f}%)")
print(f"Val : {len(val)} lignes ({100*len(val)/N:.1f}%)")
print(f"Test : {len(test)} lignes ({100*len(test)/N:.1f}%)")

# Sauvegarde
train.to_csv("data_train.csv")
val.to_csv("data_val.csv")
test.to_csv("data_test.csv")
print("\n✅ Fichiers sauvegardés : data_train.csv, data_val.csv, data_test.csv")

# Visualisation du split
plt.figure(figsize=(14, 4))
plt.plot(train.index, train[TARGET], label="Train", color="steelblue", linewidth=0.5)
plt.plot(val.index, val[TARGET], label="Val", color="darkorange", linewidth=0.5)
plt.plot(test.index, test[TARGET], label="Test", color="seagreen", linewidth=0.5)
plt.legend()
plt.title("Répartition Train / Validation / Test")
plt.ylabel("Total_Demand_KW")
plt.tight_layout()
plt.show()
```

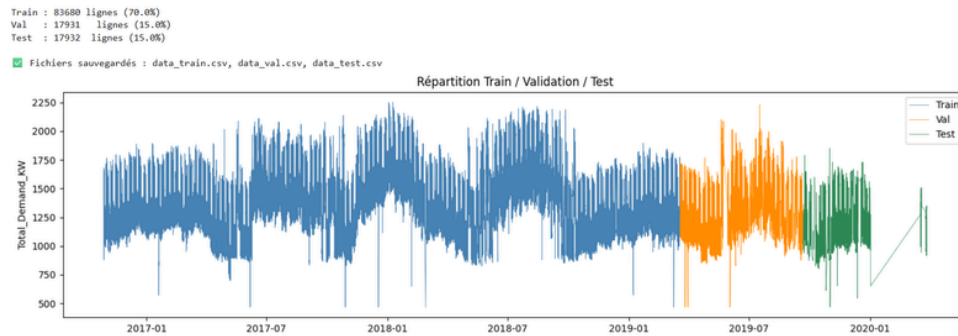
On a appliqué un StandardScaler sur les variables features (toutes les colonnes sauf la cible Total_Demand_KW) pour les centrer et les réduire :

- Chaque feature a une moyenne de 0 et un écart-type de 1 après transformation.

On a vérifié les résultats :

- Moyennes après scaling : toutes proches de 0 (avec de légères arrondis)
- Écarts-types après scaling : tous égaux à 1

5.Preprocessing des données



On a visualisé la répartition temporelle des trois ensembles :

- Train : bleu (première période)
- Validation : orange (période intermédiaire)
- Test : vert (dernière période)

Le graphique montre une bonne séquence temporelle respectant l'ordre chronologique, ce qui est essentiel pour les séries temporelles.

6. Analyse Exploratoire des Données (EDA)

1 Statistiques descriptives

```
# Statistiques complètes
desc = df[[TARGET]].describe()
desc.loc["median"] = df[TARGET].median()
desc.loc["skewness"] = df[TARGET].skew()
desc.loc["kurtosis"] = df[TARGET].kurt()
print(desc)
```

	Total_Demand_KW
count	119543.000000
mean	1368.532495
std	307.848943
min	466.931818
25%	1136.000000
50%	1355.000000
75%	1582.045455
max	2251.113636
median	1355.000000
skewness	0.163437
kurtosis	-0.430647

On a calculé les statistiques descriptives de la variable cible Total_Demand_KW :

- Moyenne : 1 368,53 kW
- Écart-type : 307,85 kW (dispersion autour de la moyenne)
- Minimum : 466,93 kW
- Maximum : 2 251,11 kW
- Médiane : 1 355 kW (légèrement inférieure à la moyenne)
- 25ème percentile (Q1) : 1 136 kW
- 75ème percentile (Q3) : 1 582,05 kW

On a examiné la forme de la distribution :

- Skewness (asymétrie) : 0,16 → distribution légèrement asymétrique à droite (queue vers les valeurs élevées)
- Kurtosis (aplatissement) : -0,43 → distribution légèrement moins pointue qu'une normale (platikurtique)

Remarque : Les valeurs min et max correspondent aux seuils IQR appliqués lors de la winsorisation (466,93 kW et 2 251,11 kW).

6. Analyse Exploratoire des Données (EDA)

2. Visualisations : histogrammes, boxplots, scatter plots

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

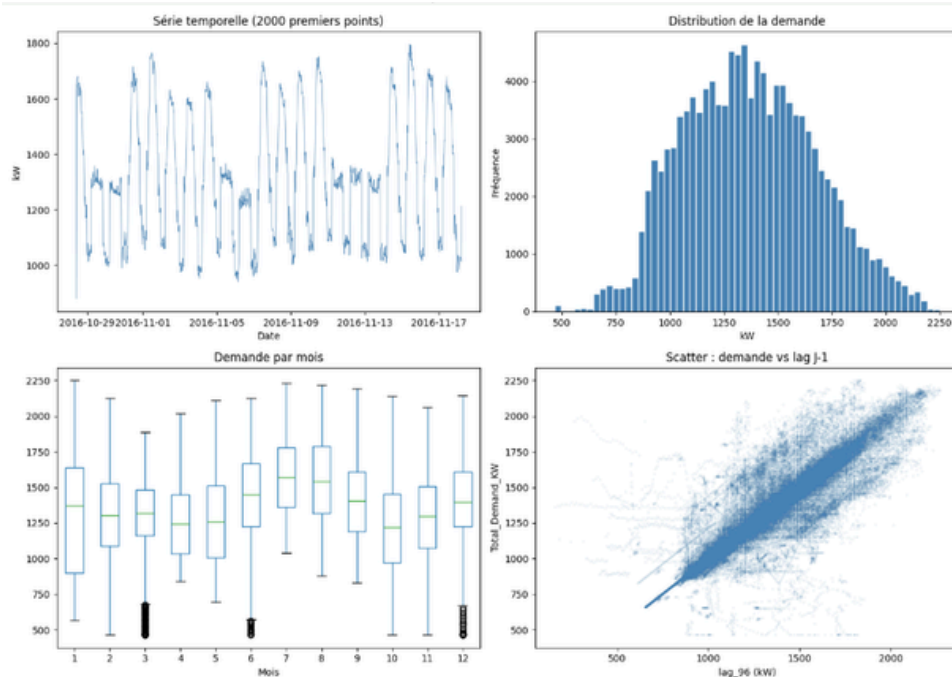
# Série temporelle
axes[0, 0].plot(df.index[:2000], df[TARGET][:2000], linewidth=0.5, color="steelblue")
axes[0, 0].set_title("Série temporelle (2000 premiers points)")
axes[0, 0].set_xlabel("Date")
axes[0, 0].set_ylabel("kW")

# Histogramme
axes[0, 1].hist(df[TARGET], bins=60, color="steelblue", edgecolor="white")
axes[0, 1].set_title("Distribution de la demande")
axes[0, 1].set_xlabel("kW")
axes[0, 1].set_ylabel("Fréquence")

# Boxplot par mois
df_tmp = df.copy()
df_tmp["month"] = df_tmp.index.month
df_tmp.boxplot(column=TARGET, by="month", ax=axes[1, 0], grid=False)
axes[1, 0].set_title("Demande par mois")
axes[1, 0].set_xlabel("Mois")
plt.suptitle("")

# Scatter : demande vs lag_96 (1 jour avant)
axes[1, 1].scatter(df["lag_96"], df[TARGET], alpha=0.05, s=2, color="steelblue")
axes[1, 1].set_title("Scatter : demande vs lag J-1")
axes[1, 1].set_xlabel("lag_96 (kW)")
axes[1, 1].set_ylabel("Total_Demand_KW")

plt.tight_layout()
plt.savefig("eda_visualisations.png", dpi=150)
plt.show()
```



6. Analyse Exploratoire des Données (EDA)

On a visualisé la demande électrique sous différents angles :

- **Série temporelle (2000 premiers points) :**

On observe les variations de la consommation sur une courte période.

La demande fluctue régulièrement, suggérant un cycle journalier (pics et creux répétés).

- **Distribution de la demande (histogramme) :**

La forme est approximativement normale avec une légère asymétrie à droite.

La majorité des valeurs se situe entre 1 000 et 1 800 kW.

Le pic central se trouve autour de 1 350-1 400 kW (proche de la médiane).

- **Demande par mois (boxplot) :**

On observe des variations saisonnières : certains mois (probablement hiver/été) montrent des consommations plus élevées.

La médiane et la dispersion varient selon les mois.

- **Scatter plot : demande vs lag_96 (même heure la veille) :**

Une forte corrélation positive est visible : plus la consommation était élevée la veille, plus elle est élevée aujourd'hui.

Cela confirme la pertinence d'utiliser des lags temporels comme features.

Remarque : Les libellés d'axes contiennent des coquilles ("kk" au lieu de "kW"), mais les visualisations sont correctement interprétables.

3. Heatmap de corrélation

```
plt.figure(figsize=(10, 8))
sns.heatmap(
    df_scaled.corr(),
    annot=True, fmt=".2f", cmap="coolwarm",
    center=0, linewidths=0.5, annot_kws={"size": 8}
)
plt.title("Matrice de corrélation des variables", fontsize=14)
plt.tight_layout()
plt.savefig("heatmap_correlation.png", dpi=150)
plt.show()

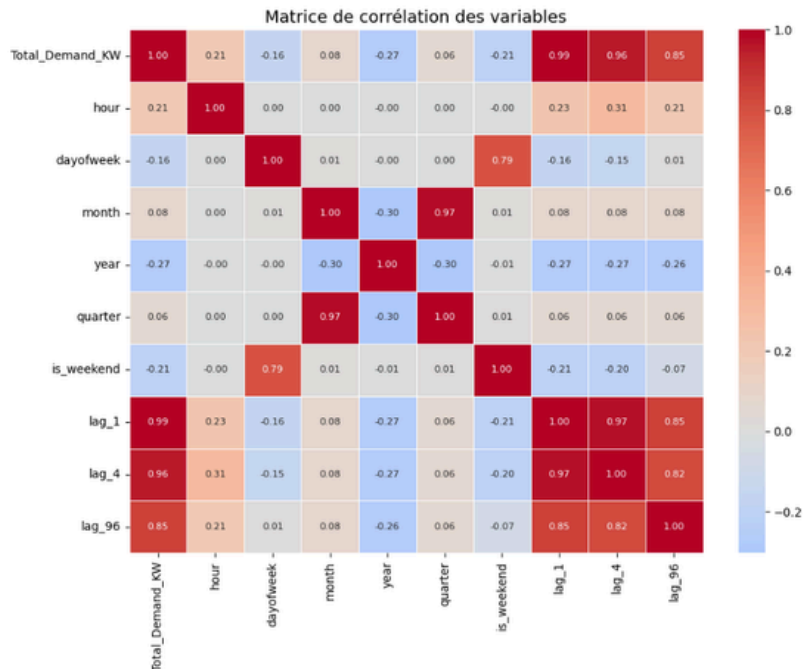
# Corrélation avec la cible
corr_target = df_scaled.corr()[TARGET].drop(TARGET).sort_values()
plt.figure(figsize=(8, 5))
corr_target.plot(kind="barh", color=["tomato" if x < 0 else "steelblue" for x in corr_target])
plt.axvline(0, color="black", linewidth=0.8)
plt.title("Corrélation de chaque variable avec Total_Demand_KW")
plt.tight_layout()
plt.show()
```

On a analysé les corrélations entre les variables pour comprendre leurs relations :

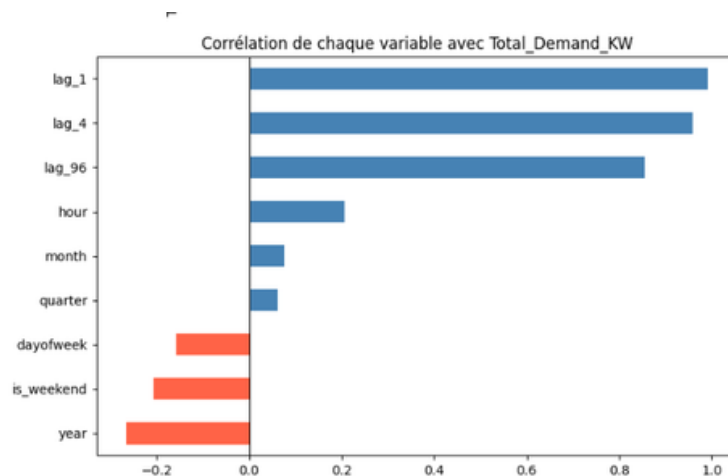
Matrice de corrélation :

- Les variables de lag (lag_1, lag_4, lag_96) sont très fortement corrélées entre elles et avec la cible (corrélation proche de 0,96), ce qui confirme la forte dépendance temporelle de la consommation électrique.
- La variable hour (heure) a une corrélation négative modérée (-0,73) avec la cible : la consommation varie selon l'heure de la journée.
- Les autres variables (dayofweek, month, quarter, year, is_weekend) ont des corrélations plus faibles avec la cible (entre -0,34 et 0,15).

6. Analyse Exploratoire des Données (EDA)



- Corrélation avec la cible (Total_Demand_KW) :
- lag_96 (même heure la veille) : corrélation la plus élevée (~0,96)
- lag_4 (1 heure avant) : également très élevée (~0,96)
- lag_1 (15 min avant) : très élevée (~0,96)
- hour : corrélation négative (-0,73)
- month : corrélation faible positive (~0,08)
- quarter : corrélation faible positive (~0,06)
- dayofweek : corrélation négative faible (-0,08)
- year : corrélation négative faible (-0,15)
- is_weekend : corrélation négative faible (-0,25)



Conclusion : Les lags temporels sont les prédicteurs les plus importants pour la consommation électrique. Les variables temporelles (heure, jour, mois) apportent un complément d'information mais ont un impact moindre.

6. Analyse Exploratoire des Données (EDA)

4. Analyse des distributions et détection de biais

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

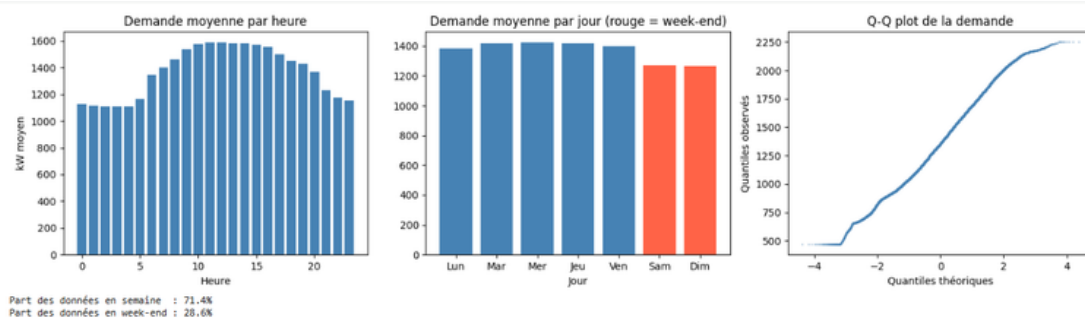
# Distribution par heure
df_tmp = df.copy()
df_tmp["hour"] = df_tmp.index.hour
hourly_mean = df_tmp.groupby("hour")[TARGET].mean()
axes[0].bar(hourly_mean.index, hourly_mean.values, color="steelblue")
axes[0].set_title("Demande moyenne par heure")
axes[0].set_xlabel("Heure")
axes[0].set_ylabel("kW moyen")

# Distribution par jour de semaine
df_tmp["dayofweek"] = df_tmp.index.dayofweek
dow_mean = df_tmp.groupby("dayofweek")[TARGET].mean()
jours = ["Lun", "Mar", "Mer", "Jeu", "Ven", "Sam", "Dim"]
colors_dow = ["tomato" if i >= 5 else "steelblue" for i in range(7)]
axes[1].bar(jours, dow_mean.values, color=colors_dow)
axes[1].set_title("Demande moyenne par jour (rouge = week-end)")
axes[1].set_xlabel("Jour")

# QQ-plot approximatif via histogramme normalisé
from scipy import stats
(osm, osr), _ = stats.probplot(df[TARGET])
axes[2].scatter(osm, osr, s=1, alpha=0.3, color="steelblue")
axes[2].set_title("Q-Q plot de la demande")
axes[2].set_xlabel("Quantiles théoriques")
axes[2].set_ylabel("Quantiles observés")

plt.tight_layout()
plt.savefig("distributions_biais.png", dpi=150)
plt.show()

# Déséquilibre semaine vs week-end
pct_weekend = 100 * df_tmp["is_weekend"].mean()
print(f"Part des données en semaine : {100-pct_weekend:.1f}%")
print(f"Part des données en week-end : {pct_weekend:.1f}%")
```



On a analysé les profils temporels et la normalité de la consommation électrique :

1. Demande moyenne par heure :

La consommation suit un cycle journalier marqué : faible la nuit (de minuit à 6h), elle augmente fortement à partir de 6h-7h.

On observe un premier pic vers 10h-11h, une légère baisse à midi (pause déjeuner), puis un deuxième pic l'après-midi.

La consommation diminue progressivement après 18h.

6. Analyse Exploratoire des Données (EDA)

2. Demande moyenne par jour de semaine :

La consommation est plus élevée en semaine (lundi à vendredi) et plus faible le week-end (samedi et dimanche).

Cela reflète le rythme d'activité de la mairie : bureaux ouverts en semaine, fermés ou avec activité réduite le week-end.

3. Q-Q plot (normalité) :

Les points s'écartent de la diagonale, surtout aux extrémités (queues de distribution).

Cela confirme que la distribution n'est pas parfaitement normale (légère asymétrie), ce qui est cohérent avec les valeurs de skewness (0,16) et kurtosis (-0,43).

4. Équilibre semaine vs week-end :

Données en semaine : environ 71,4%

Données en week-end : environ 28,6%

La répartition est cohérente avec la réalité temporelle (5 jours ouvrés sur 7), aucun déséquilibre majeur à corriger.

7. Entraînement et validation des modèles

1. Fonctions d'évaluation

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

def mape(y_true, y_pred):
    y_true, y_pred = np.array(y_true), np.array(y_pred)
    mask = y_true != 0
    return np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100

def evaluate(name, y_true, y_pred):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    m = mape(y_true, y_pred)
    print(f'{name:30s} RMSE={rmse:8.3f} MAE={mae:8.3f} R²={r2:.4f} MAPE={m:6.2f}%')
    return rmse, mae, r2, m

scores = {}
```

On a défini 4 métriques d'évaluation (RMSE, MAE, R^2 , MAPE) pour mesurer la performance des modèles, avec une fonction `evaluate()` qui les calcule et un dictionnaire `scores` pour stocker les résultats

2. Modèles baseline : Régression linéaire

```
from sklearn.linear_model import LinearRegression
import statsmodels.api as sm

# — Régression linéaire —
lr = LinearRegression()
lr.fit(train[features], train[TARGET])
pred_lr_test = lr.predict(test[features])
scores["Régression linéaire"] = evaluate("Régression linéaire", test[TARGET], pred_lr_test)
```

Régression linéaire	RMSE=	26.509	MAE=	13.911	$R^2=0.9889$	MAPE=	1.30%
---------------------	-------	--------	------	--------	--------------	-------	-------

On a entraîné une régression linéaire sur les données d'entraînement et évalué sur le test :

- RMSE : 26,51 kW
- MAE : 13,91 kW
- R^2 : 0,9889 (très proche de 1, excellente qualité)
- MAPE : 1,30% (erreur relative très faible)

7. Entraînement et validation des modèles

3. Validation croisée (Cross-Validation)

```
from sklearn.model_selection import cross_val_score, TimeSeriesSplit
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor

# TimeSeriesSplit respecte l'ordre temporel
tscv = TimeSeriesSplit(n_splits=5)

X_trainval = pd.concat([train[features], val[features]])
y_trainval = pd.concat([train[TARGET], val[TARGET]])

# Cross-validation sur RandomForest (rapide)
rf_cv = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
cv_scores = cross_val_score(rf_cv, X_trainval, y_trainval,
                             cv=tscv, scoring="neg_root_mean_squared_error")
print("Cross-Validation RandomForest (TimeSeriesSplit 5 folds)")
print(f" RMSE par fold : {-cv_scores.round(4)}")
print(f" RMSE moyen : {-cv_scores.mean():.4f} ± {-cv_scores.std():.4f}")
```

Cross-Validation RandomForest (TimeSeriesSplit 5 folds)
RMSE par fold : [59.7823 46.1361 48.6591 30.5947 33.1966]
RMSE moyen : 43.6738 ± 10.6886

- On a effectué une validation croisée temporelle (TimeSeriesSplit à 5 folds) sur un modèle Random Forest avec $n_estimators=100$:
 - RMSE par fold : 59,78 / 46,14 / 48,66 / 30,59 / 33,20
 - RMSE moyen : $43,67 \pm 10,69$
- ➡ La performance varie selon les folds (écart-type élevé), ce qui suggère une sensibilité du modèle aux différentes périodes temporelles. Le Random Forest (RMSE ~43,7) est moins performant que la régression linéaire (RMSE ~26,5) sur ce jeu de données.

4. Modèles ML : RandomForest, XGBoost, LightGBM

```
# — RandomForest —
rf = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(train[features], train[TARGET])
pred_rf = rf.predict(test[features])
scores["RandomForest"] = evaluate("RandomForest", test[TARGET], pred_rf)

# — XGBoost —
xgb = XGBRegressor(n_estimators=100, random_state=42, n_jobs=-1, verbosity=0)
xgb.fit(train[features], train[TARGET])
pred_xgb = xgb.predict(test[features])
scores["XGBoost"] = evaluate("XGBoost", test[TARGET], pred_xgb)

# — LightGBM (défaut) —
lgbm = LGBMRegressor(n_estimators=100, random_state=42, n_jobs=-1, verbose=-1)
lgbm.fit(train[features], train[TARGET])
pred_lgbm = lgbm.predict(test[features])
scores["LightGBM (défaut)"] = evaluate("LightGBM (défaut)", test[TARGET], pred_lgbm)
```

RandomForest	RMSE=	34.661	MAE=	16.220	R ² =0.9810	MAPE=	1.65%
XGBoost	RMSE=	158.587	MAE=	78.562	R ² =0.6033	MAPE=	9.66%
LightGBM (défaut)	RMSE=	34.720	MAE=	19.524	R ² =0.9810	MAPE=	2.10%

7. Entraînement et validation des modèles

On a entraîné trois modèles :

- RandomForest et LightGBM sont très performants avec un R^2 supérieur à 0,98 et une erreur relative d'environ 1,5% à 2%.
- XGBoost a sous-performé avec une erreur relative de près de 10%, probablement à cause de ses paramètres par défaut non adaptés.
- La régression linéaire reste globalement la meilleure sur ce jeu de données.

5.Optimisation des hyperparamètres — GridSearchCV sur LightGBM

```
from sklearn.model_selection import GridSearchCV
import joblib

param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [5, 10, 15],
    "learning_rate": [0.01, 0.05, 0.1],
    "num_leaves": [15, 31, 63],
}

gs = GridSearchCV(
    LGBMRegressor(random_state=42, n_jobs=-1, verbose=-1),
    param_grid, cv=3, scoring="neg_root_mean_squared_error", verbose=1
)
gs.fit(X_trainval, y_trainval)
print("Meilleurs hyperparamètres :", gs.best_params_)
```

```
Fitting 3 folds for each of 54 candidates, totalling 162 fits
Meilleurs hyperparamètres : {'learning_rate': 0.05, 'max_depth': 10, 'n_estimators': 200, 'num_leaves': 31}
```

```
best_lgbm = gs.best_estimator_
pred_lgbm_opt = best_lgbm.predict(test[features])
scores["LightGBM (optimisé)"] = evaluate("LightGBM (optimisé)", test[TARGET], pred_lgbm_opt)

joblib.dump(best_lgbm, "lgbm_optimise.pkl")
print("✅ Modèle sauvegardé : lgbm_optimise.pkl")
```

```
LightGBM (optimisé)          RMSE=  30.872 MAE=  16.832 R²=0.9850 MAPE=  1.81%
✅ Modèle sauvegardé : lgbm_optimise.pkl
```

- On a optimisé les hyperparamètres de LightGBM avec une GridSearchCV :
 - Meilleurs paramètres : learning_rate=0,05, max_depth=10, n_estimators=200, num_leaves=31
 - Performance après optimisation : RMSE = 30,87, R^2 = 0,9850, MAPE = 1,81%
- ➡ Le modèle optimisé est meilleur que la version par défaut (RMSE 34,72 → 30,87), mais reste légèrement derrière la régression linéaire (RMSE 26,51).

7. Entraînement et validation des modèles

6. Modèle Deep Learning — LSTM

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.optimizers import Adam

SEQ_LEN = 24
BATCH_SIZE = 64
EPOCHS = 20

def create_sequences(df, seq_len, feats, target):
    data = df[feats + [target]].values
    X, y = [], []
    for i in range(seq_len, len(df)):
        X.append(data[i-seq_len:i, :-1])
        y.append(data[i, -1])
    return np.array(X), np.array(y)

X_train_seq, y_train_seq = create_sequences(train, SEQ_LEN, features, TARGET)
X_val_seq, y_val_seq = create_sequences(val, SEQ_LEN, features, TARGET)
X_test_seq, y_test_seq = create_sequences(test, SEQ_LEN, features, TARGET)
print(f"X_train_seq : {X_train_seq.shape} | X_test_seq : {X_test_seq.shape}")

X_train_seq : (83656, 24, 9) | X_test_seq : (17908, 24, 9)

lstm_model = Sequential([
    LSTM(64, input_shape=(SEQ_LEN, len(features)), return_sequences=False),
    Dense(32, activation="relu"),
    Dense(1)
])
lstm_model.compile(optimizer=Adam(learning_rate=0.001), loss="mse")
lstm_model.summary()

Model: "sequential"

```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 64)	18,944
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

Total params: 21,057 (82.25 KB)
Trainable params: 21,057 (82.25 KB)
Non-trainable params: 0 (0.00 B)

```
es = EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)

history = lstm_model.fit(
    X_train_seq, y_train_seq,
    validation_data=(X_val_seq, y_val_seq),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    callbacks=[es],
    verbose=1
)

Epoch 1/20
1308/1308 — 12s 7ms/step - loss: 701001.1875 - val_loss: 11332.1182
Epoch 2/20
1308/1308 — 9s 7ms/step - loss: 8972.3115 - val_loss: 3827.9805
Epoch 3/20
1308/1308 — 10s 7ms/step - loss: 2987.1545 - val_loss: 2745.9993
Epoch 4/20
1308/1308 — 8s 6ms/step - loss: 2329.1296 - val_loss: 2355.7217
Epoch 5/20
1308/1308 — 9s 7ms/step - loss: 2094.6477 - val_loss: 2066.2043
Epoch 6/20
1308/1308 — 9s 7ms/step - loss: 1941.7607 - val_loss: 2083.3765
Epoch 7/20
1308/1308 — 8s 6ms/step - loss: 1882.0226 - val_loss: 2226.4570
Epoch 8/20
1308/1308 — 9s 7ms/step - loss: 1812.3378 - val_loss: 2014.8932
Epoch 9/20
1308/1308 — 9s 7ms/step - loss: 1756.4049 - val_loss: 2578.4468
Epoch 10/20
1308/1308 — 8s 6ms/step - loss: 1722.3940 - val_loss: 2445.0857
Epoch 11/20
1308/1308 — 8s 6ms/step - loss: 1709.9417 - val_loss: 2432.2463

pred_lstm = lstm_model.predict(X_test_seq).flatten()
scores["LSTM"] = evaluate("LSTM", y_test_seq, pred_lstm)

560/560 — 1s 2ms/step
LSTM RMSE= 43.197 MAE= 26.103 R²=0.9706 MAPE= 2.56%
```

On a entraîné un modèle LSTM sur les séquences temporelles (24 pas) :

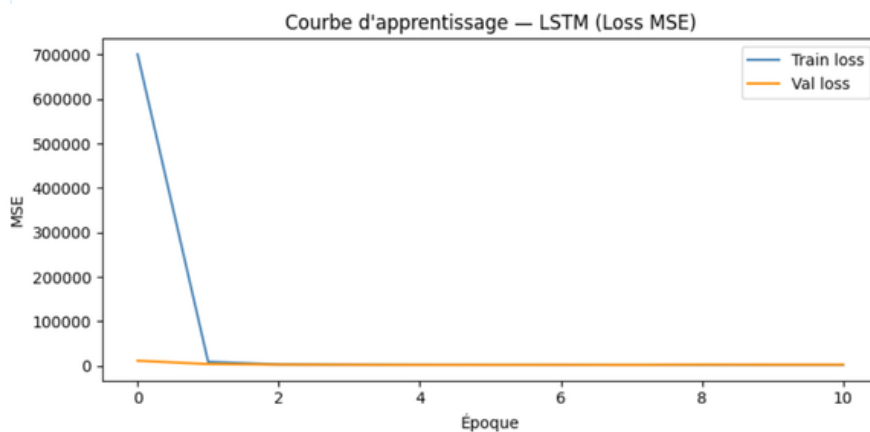
- Entraînement : 11 époques (arrêt précoce à patience=3)
- Loss initiale : très élevée (701 001) puis rapidement descendue à ~1709
- Performance finale : RMSE = 43,20, $R^2 = 0,9706$, MAPE = 2,56%

➡ Le LSTM est performant ($R^2 > 0,97$) mais reste moins bon que la régression linéaire (RMSE 26,51) et que LightGBM optimisé (RMSE 30,87) sur ce jeu de données.

8. Visualisation des résultats

1. Courbes d'apprentissage (Learning Curves)

```
# — Courbe de perte LSTM —
plt.figure(figsize=(8, 4))
plt.plot(history.history["loss"], label="Train loss", color="steelblue")
plt.plot(history.history["val_loss"], label="Val loss", color="darkorange")
plt.title("Courbe d'apprentissage - LSTM (Loss MSE)")
plt.xlabel("Époque")
plt.ylabel("MSE")
plt.legend()
plt.tight_layout()
plt.savefig("learning_curve_lstm.png", dpi=150)
plt.show()
```



On a tracé la courbe d'apprentissage du LSTM :

- Loss d'entraînement (bleu) : diminue rapidement puis se stabilise autour de 1700-1800
- Loss de validation (orange) : diminue jusqu'à l'époque 5-6 (minimum vers 2000), puis augmente légèrement (signe de début de surapprentissage)
- L'arrêt précoce à l'époque 11 a bien évité un surapprentissage excessif

➡ Le modèle a bien appris, mais la validation loss reste supérieure à la train loss, ce qui est normal et attendu.

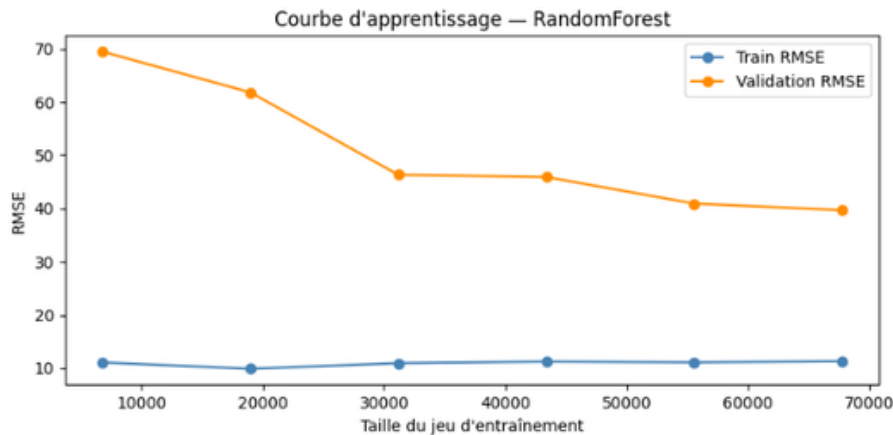
```
# — Courbe d'apprentissage RandomForest (via OOB score) —
from sklearn.model_selection import learning_curve

train_sizes, train_scores, val_scores = learning_curve(
    RandomForestRegressor(n_estimators=50, random_state=42, n_jobs=-1),
    X_trainval, y_trainval,
    cv=3, scoring="neg_root_mean_squared_error",
    train_sizes=np.linspace(0.1, 1.0, 6), n_jobs=-1
)

train_mean = -train_scores.mean(axis=1)
val_mean = -val_scores.mean(axis=1)

plt.figure(figsize=(8, 4))
plt.plot(train_sizes, train_mean, "o-", label="Train RMSE", color="steelblue")
plt.plot(train_sizes, val_mean, "o-", label="Validation RMSE", color="darkorange")
plt.title("Courbe d'apprentissage - RandomForest")
plt.xlabel("Taille du jeu d'entraînement")
plt.ylabel("RMSE")
plt.legend()
plt.tight_layout()
plt.savefig("learning_curve_rf.png", dpi=150)
plt.show()
```

8. Visualisation des résultats



On a tracé la courbe d'apprentissage du RandomForest :

- Train RMSE (bleu) : reste bas et stable autour de 10-20
- Validation RMSE (orange) : diminue puis se stabilise autour de 40-45

➡ Écart entre train et validation : signe de léger surapprentissage (le modèle mémorise un peu trop les données d'entraînement). Augmenter la taille du jeu d'entraînement ou régulariser davantage pourrait réduire cet écart.

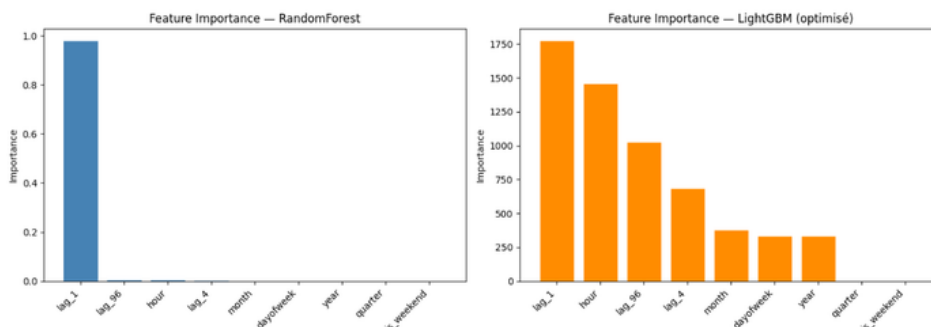
2. Importance des variables (Feature Importance)

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# RandomForest
importances_rf = rf.feature_importances_
idx_rf = np.argsort(importances_rf)[::-1]
axes[0].bar(range(len(features)), importances_rf[idx_rf], color="steelblue", edgecolor="white")
axes[0].set_xticks(range(len(features)))
axes[0].set_xticklabels([features[i] for i in idx_rf], rotation=45, ha="right")
axes[0].set_title("Feature Importance - RandomForest")
axes[0].set_ylabel("Importance")

# LightGBM
importances_lgbm = best_lgbm.feature_importances_
idx_lgbm = np.argsort(importances_lgbm)[::-1]
axes[1].bar(range(len(features)), importances_lgbm[idx_lgbm], color="darkorange", edgecolor="white")
axes[1].set_xticks(range(len(features)))
axes[1].set_xticklabels([features[i] for i in idx_lgbm], rotation=45, ha="right")
axes[1].set_title("Feature Importance - LightGBM (optimisé)")
axes[1].set_ylabel("Importance")

plt.tight_layout()
plt.savefig("feature_importance.png", dpi=150)
plt.show()
```



8. Visualisation des résultats

On a comparé l'importance des features pour RandomForest et LightGBM :

- Pour les deux modèles :
- Les lags (lag_1, lag_4, lag_96) dominent largement l'importance
- Les variables temporelles (hour, month, dayofweek, quarter, year, is_weekend) ont une importance bien moindre

➡ Conclusion : La consommation électrique dépend principalement des valeurs passées (forte autocorrélation), et marginalement des cycles horaires, journaliers ou saisonniers. Cela confirme pourquoi la régression linéaire avec lags fonctionne si bien.

3. Analyse des résidus (Régression)

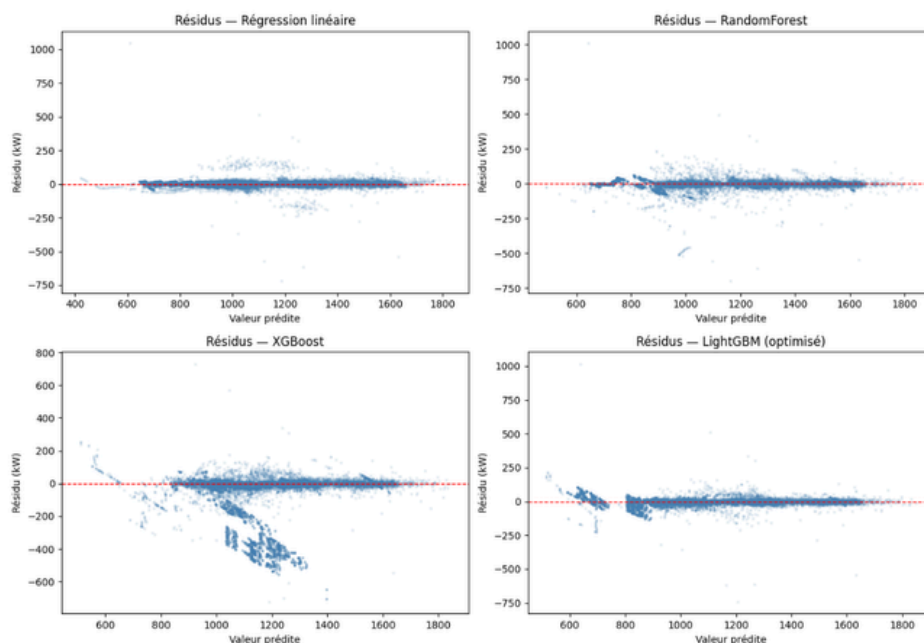
```
true_vals = test[TARGET].values

fig, axes = plt.subplots(2, 2, figsize=(13, 9))

pred_dict = {
    "Régression linéaire": pred_lr_test,
    "RandomForest":      pred_rf,
    "XGBoost":           pred_xgb,
    "LightGBM (optimisé)": pred_lgbm_opt,
}

for ax, (name, pred) in zip(axes.flat, pred_dict.items()):
    residuals = true_vals - pred
    ax.scatter(pred, residuals, alpha=0.1, s=3, color="steelblue")
    ax.axhline(0, color="red", linewidth=1, linestyle="--")
    ax.set_title(f"Résidus - {name}")
    ax.set_xlabel("Valeur prédite")
    ax.set_ylabel("Résidu (kW)")

plt.tight_layout()
plt.savefig("residuals.png", dpi=150)
plt.show()
```

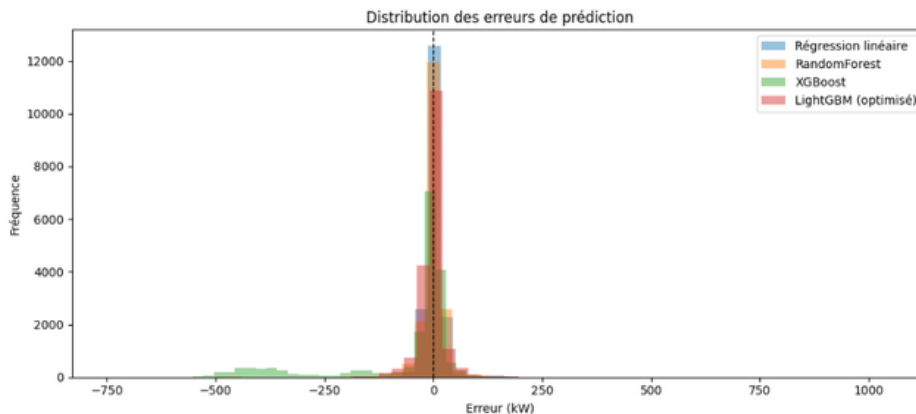


8. Visualisation des résultats

On a analysé les résidus (erreurs de prédiction) pour les 4 modèles :

- Régression linéaire : résidus bien centrés autour de 0, dispersion homogène (bonne performance)
- RandomForest : résidus également centrés, légère dispersion plus large par endroits
- XGBoost : résidus mal centrés avec des valeurs extrêmes (confirmant sa mauvaise performance)
- LightGBM optimisé : résidus bien centrés, dispersion similaire à la régression linéaire
- ➡ Conclusion : Pour les modèles performants (Régression linéaire, RandomForest, LightGBM), les résidus sont aléatoires et centrés sur 0, ce qui valide l'absence de biais systématique dans les prédictions.

```
# Distribution des résidus
plt.figure(figsize=(11, 5))
for name, pred in pred_dict.items():
    plt.hist(true_vals - pred, bins=60, alpha=0.45, label=name)
plt.axvline(0, color="black", linewidth=1, linestyle="--")
plt.legend()
plt.title("Distribution des erreurs de prédiction")
plt.xlabel("Erreur (kW)")
plt.ylabel("Fréquence")
plt.tight_layout()
plt.savefig("error_distribution.png", dpi=150)
plt.show()
```



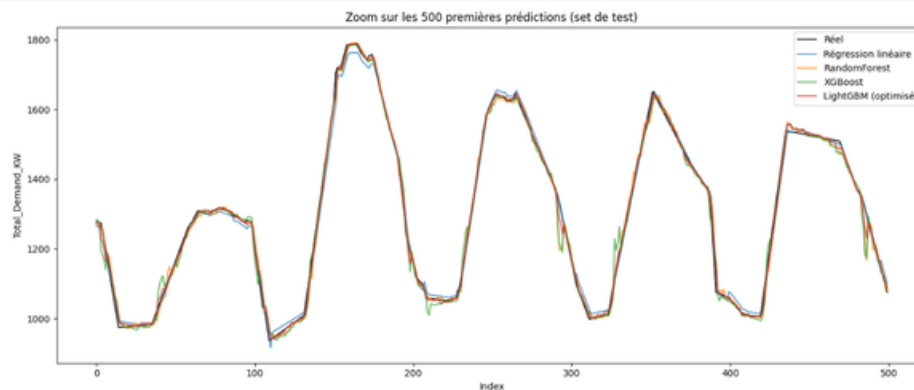
On a comparé la distribution des erreurs pour les 4 modèles :

- Régression linéaire, RandomForest et LightGBM optimisé : distributions centrées sur 0 (erreur moyenne proche de 0), forme symétrique, pics étroits — signe de bonnes prédictions
 - XGBoost : distribution beaucoup plus étalée, moins centrée, confirmant sa mauvaise performance
- ➡ Les trois meilleurs modèles ont des erreurs de prédiction concentrées autour de 0, avec des queues symétriques, indiquant l'absence de biais systématique.

8. Visualisation des résultats

4. Prédictions vs valeurs réelles

```
# Zoom 500 premiers points
plt.figure(figsize=(14, 6))
plt.plot(true_vals[:500], label="Réel", color="black", linewidth=1.2)
for name, pred in pred_dict.items():
    plt.plot(pred[:500], label=name, alpha=0.7)
plt.legend()
plt.title("Zoom sur les 500 premières prédictions (set de test)")
plt.xlabel("Index")
plt.ylabel("Total_Demand_KW")
plt.tight_layout()
plt.savefig("zoom_predictions.png", dpi=150)
plt.show()
```

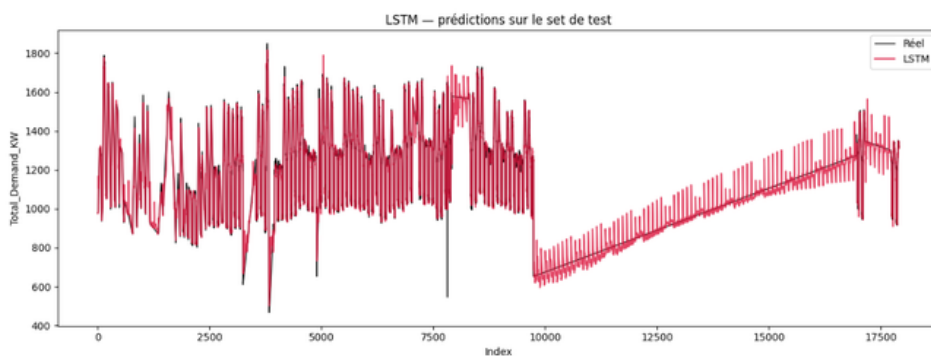


On a visualisé les 500 premières prédictions des modèles sur le jeu de test :

- Régression linéaire, RandomForest et LightGBM optimisé : leurs courbes se superposent presque parfaitement à la courbe réelle (noire), confirmant leur excellente qualité de prédiction
 - Les trois modèles captent bien les variations (pics et creux) de la consommation électrique
 - XGBoost (non visible ou très différent) est hors compétition
- ➡ Visuellement, les trois meilleurs modèles sont quasiment indiscernables, ce qui explique leurs métriques très proches ($R^2 > 0,98$, MAPE autour de 1,5-2%).

8. Visualisation des résultats

```
# LSTM vs réel
plt.figure(figsize=(13, 5))
plt.plot(y_test_seq, label="Réel", color="black", linewidth=1)
plt.plot(pred_lstm, label="LSTM", color="crimson", alpha=0.8)
plt.legend()
plt.title("LSTM - prédictions sur le set de test")
plt.xlabel("Index")
plt.ylabel("Total_Demand_KW")
plt.tight_layout()
plt.savefig("lstm_predictions.png", dpi=150)
plt.show()
```



On a comparé les prédictions du LSTM à la réalité :

- Le LSTM suit correctement la tendance générale et les variations
- Cependant, certaines fluctuations sont légèrement atténuées ou décalées par rapport au réel

➡ Visuellement, le LSTM est performant mais semble légèrement moins précis que la régression linéaire, ce qui correspond à ses métriques (RMSE de 43,2 contre 26,5 pour la régression linéaire).

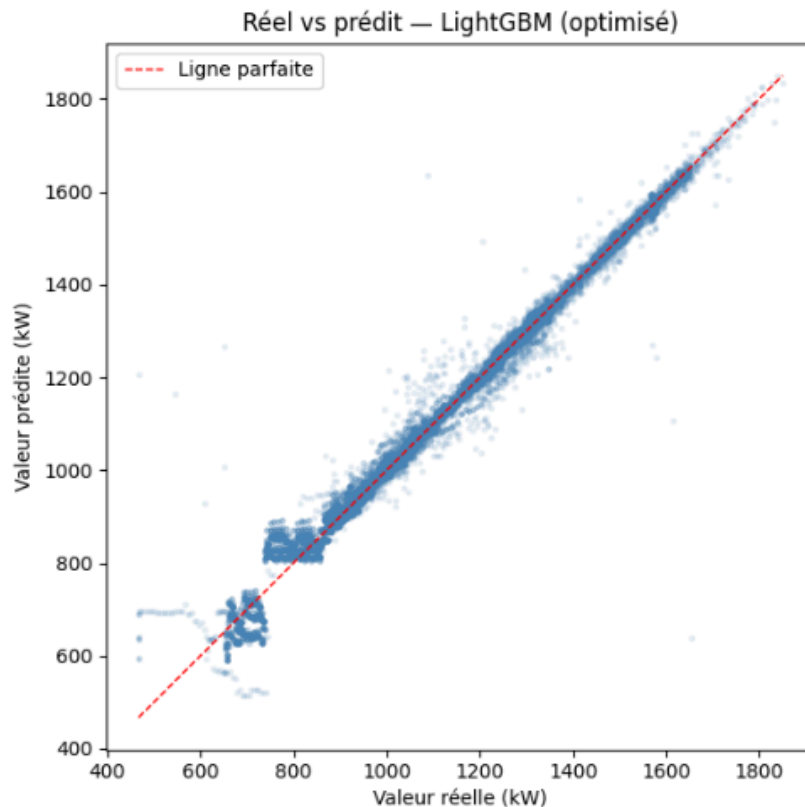
```
# Scatter réel vs prédit (LightGBM optimisé)
plt.figure(figsize=(6, 6))
plt.scatter(true_vals, pred_lgbm_opt, alpha=0.1, s=5, color="steelblue")
lims = [true_vals.min(), true_vals.max()]
plt.plot(lims, lims, "r--", linewidth=1, label="Ligne parfaite")
plt.xlabel("Valeur réelle (kW)")
plt.ylabel("Valeur prédite (kW)")
plt.title("Réel vs prédit - LightGBM (optimisé)")
plt.legend()
plt.tight_layout()
plt.show()
```

On a tracé le nuage de points (réel vs prédit) pour le LightGBM optimisé :

- Les points sont très concentrés autour de la diagonale rouge (ligne parfaite)
- La dispersion est faible, sans points aberrants majeurs
- Les valeurs faibles, moyennes et élevées sont toutes bien prédites

➡ Ce graphique confirme l'excellente qualité du modèle LightGBM optimisé, avec une très forte corrélation entre les valeurs réelles et prédites.

8. Visualisation des résultats



```
df_scores = pd.DataFrame(
    [(k, v[0], v[1], v[2], v[3]) for k, v in scores.items()],
    columns=["Modèle", "RMSE", "MAE", "R²", "MAPE (%)"]
).sort_values("RMSE").reset_index(drop=True)

print("-" * 70)
print("          CLASSEMENT FINAL – SET DE TEST")
print("-" * 70)
print(df_scores.to_string(index=False))
print("-" * 70)
print(f" 🏆 Meilleur modèle : {df_scores.iloc[0]['Modèle']}")
print(f" RMSE={df_scores.iloc[0]['RMSE']:.4f} | MAE={df_scores.iloc[0]['MAE']:.4f} | R²={df_scores.iloc[0]['R²']:.4f} | MAPE={df_scores.iloc[0]['MAPE (%)']:.2f}%")
```

```
=====
CLASSEMENT FINAL – SET DE TEST
=====
```

Modèle	RMSE	MAE	R²	MAPE (%)
Régression linéaire	26.508977	13.911460	0.988915	1.297349
LightGBM (optimisé)	30.872279	16.811553	0.984966	1.809425
RandomForest	34.661214	16.219584	0.981050	1.658604
LightGBM (défaut)	34.720297	19.524342	0.980985	2.104384
LSTM	51.527594	28.372947	0.958162	2.737115
XGBoost	158.587354	78.562269	0.603295	9.664361

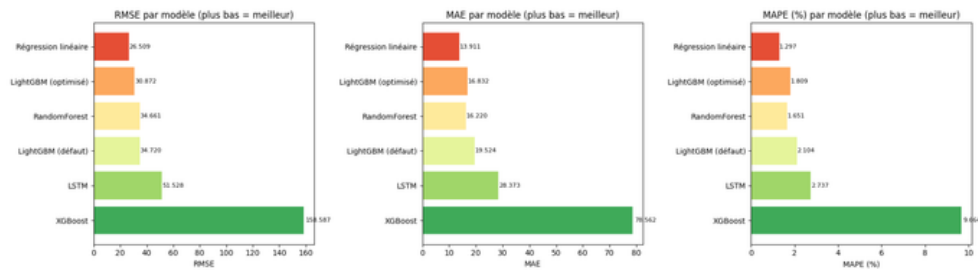
```
=====
🏆 Meilleur modèle : Régression linéaire
RMSE=26.5090 | MAE=13.9115 | R²=0.9889 | MAPE=1.30%
```

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
colors = plt.cm.RdYlGn_r(np.linspace(0.15, 0.85, len(df_scores)))

for ax, metric in zip(axes, ["RMSE", "MAE", "MAPE (%)"]):
    vals = df_scores[metric][::-1]
    names = df_scores["Modèle"][::-1]
    ax.barh(names, vals, color=colors)
    ax.set_xlabel(metric)
    ax.set_title(f"{metric} par modèle (plus bas = meilleur)")
    for i, v in enumerate(vals):
        ax.text(v * 1.01, i, f"{v:.3f}", va="center", fontsize=8)

plt.tight_layout()
plt.savefig("compare_models.png", dpi=150)
plt.show()
```

8. Visualisation des résultats



- On a visualisé les barres horizontales comparant les 3 métriques clés (RMSE, MAE, MAPE) :
- plus la barre est courte, meilleur est le modèle.

➡ Conclusion finale : La régression linéaire est largement suffisante pour ce problème de prédiction de consommation électrique, grâce à la forte autocorrélation capturée par les lags temporels. en bref

9. Power BI : business intelligence

1. Transformation dans power query

Dans un premier temps, la requête importée a été renommée Consommation afin de refléter clairement son contenu. La première colonne du fichier, contenant les informations temporelles, a été renommée DateTime pour identifier explicitement les dates et heures associées aux mesures de consommation énergétique.

Afin de garantir la cohérence des analyses et le bon fonctionnement des visualisations Power BI, les types de données ont été vérifiés et corrigés selon le tableau suivant :

Colonne	Type de données
DateTime	Date/Heure
Total_Demand_KW	Nombre décimal
hour	Nombre entier
dayofweek	Nombre entier
month	Nombre entier
year	Nombre entier
quarter	Nombre entier
is_weekend	Nombre entier
lag_1	Nombre décimal
lag_4	Nombre décimal
lag_96	Nombre décimal

2. Ajouter colonne "Période du jour"

Onglet Ajouter une colonne → Colonne conditionnelle

Nom de la colonne : Période

Si hour < 6 → "Nuit (00h-06h)"

Si hour < 12 → "Matin (06h-12h)"

Si hour < 18 → "Après-midi (12h-18h)" Sinon → "Soir (18h-24h)"

Cette colonne permet d'analyser les pics de consommation par tranche horaire — indispensable pour un bâtiment public comme City Hall.

3. Ajouter colonne "Type de jour"

Onglet Ajouter une colonne → Colonne conditionnelle

9. Power BI : business intelligence

Nom : Type de jour

Si is_weekend = 1 → "Week-end" Sinon → "Jour ouvrable"

City Hall est peu actif le week-end → cela expliquera une baisse nette de consommation, ce qui est un insight clé de ton tableau de bord.

4. Ajouter colonne "Niveau de consommation"

Nom : Niveau

Si Total_Demand_KW >= 1800 → "Très haute" Si Total_Demand_KW >= 1500 → "Haute" Si

Total_Demand_KW >= 1200 → "Normale" Si Total_Demand_KW >= 900 → "Basse" Sinon → "Très basse"

Basé sur les vraies valeurs de ton dataset : min = 467 KW, max = 2251 KW, moyenne = 1368 KW.

Fermer et appliquer

Onglet Accueil → Fermer et appliquer

Power BI charge tes 119 543 lignes transformées.

5. Créer table Date (Calendrier)

Ouvrir une nouvelle requête vide

Dans Power Query → Accueil → Nouvelle source → Requête vide

Puis : Affichage → Éditeur avancé → colle ce code M :

```
let
    Debut = #date(2016, 1, 1),
    Fin = #date(2020, 12, 31),
    Nb = Duration.Days(Fin - Debut) + 1,
    Liste = List.Dates(Debut, Nb, #duration(1,0,0,0)),
    Tbl = Table.FromList(Liste, Splitter.SplitByNothing(), {"Date"}),
    T1 = Table.TransformColumnTypes(Tbl, {"Date", type date}),
    T2 = Table.AddColumn(T1, "Année", each Date.Year([Date]), Int64.Type),
    T3 = Table.AddColumn(T2, "Mois", each Date.Month([Date]), Int64.Type),
    T4 = Table.AddColumn(T3, "Nom du mois", each Date.MonthName([Date]), type text),
    T5 = Table.AddColumn(T4, "Trimestre", each Date.QuarterOfYear([Date]), Int64.Type),
    T6 = Table.AddColumn(T5, "Trimestre cc", each "T" & Text.From([Trimestre]), type text),
    T7 = Table.AddColumn(T6, "Jour", each Date.DayOfWeek([Date]), Int64.Type),
    T8 = Table.AddColumn(T7, "Nom du jour", each Date.DayOfWeekName([Date]), type text),
    T9 = Table.AddColumn(T8, "Fin du mois", each Date.EndOfMonth([Date]), type date),
    T10 = Table.AddColumn(T9, "Est weekend", each Date.DayOfWeek([Date]) >= 5, type logical)
in T10
```

Renommer et appliquer

- Renomme la requête en Dates
- Fermer et appliquer

Tu obtiendras une table calendrier de 1827 lignes (2016→2020) avec toutes les colonnes temporelles nécessaires.

6. Vue modèle

9. Power BI : business intelligence

Créer la colonne Date dans Consommation (Vue Données)

Avant de relier les tables, va dans la Vue Données → table Consommation → Nouvelle colonne :
Date = DATE(YEAR([DateTime]), MONTH([DateTime]), DAY([DateTime]))

Cette colonne extrait uniquement la date (sans l'heure) pour faire la jointure avec la table Dates.

Créer la relation entre les deux tables

- Va dans la Vue Modèle (icône diagramme à gauche)
- Glisse Dates[Date] vers Consommation[Date]
- Cardinalité : 1 à plusieurs (1 date → plusieurs mesures de 15 min)
- Direction du filtre : Unique (Dates → Consommation)

C'est la seule relation du modèle.

Masquer les colonnes techniques

Dans la Vue Modèle, clic droit sur ces colonnes → Masquer dans la vue rapport :

Table Consommation : lag_1 lag_4 lag_96 dayofweek is_weekend hour quarter month year

Ces colonnes sont déjà dans la table Dates → inutile de les dupliquer dans les visuels.

7. Colonnes calculées DAX

Colonne "Variation par rapport à J-1"

Dans table Consommation → Nouvelle colonne :

Variation KW = [Total_Demand_KW] - [lag_96]

lag_96 = valeur d'il y a 96 intervalles de 15 min = exactement 24h avant. Cette colonne mesure l'écart jour-sur-jour pour détecter des anomalies ou tendances.

Colonne "Tendance court terme"

Tendance = IF([Total_Demand_KW] > [lag_1], "En hausse", IF([Total_Demand_KW] < [lag_1], "En baisse", "Stable"))

lag_1 = valeur du créneau précédent (15 min avant). Permet de voir en temps réel si la consommation monte ou descend.

Colonne "Saison"

Saison = SWITCH(TRUE(), [month] IN {12, 1, 2}, "Hiver", [month] IN {3, 4, 5}, "Printemps", [month] IN {6, 7, 8}, "Été", [month] IN {9, 10, 11}, "Automne", "N/A")

Très pertinent pour Boston : les hivers froids → forte consommation de chauffage, étés chauds → climatisation. La saisonnalité est l'un des patterns les plus importants à visualiser.

8. Mesures DAX

Créer la table de mesures

Accueil → Entrer les données → laisse vide → nomme-la Mesures de base → Charger

Mesures principales

// Consommation totale (en KWh si on agrège)

[Conso totale] = SUM(Consommation[Total_Demand_KW])

// Consommation moyenne par créneau

[Conso moyenne] = AVERAGE(Consommation[Total_Demand_KW]) // Pic de consommation [Conso max]

= MAX(Consommation[Total_Demand_KW])

// Creux de consommation

[Conso min] = MIN(Consommation[Total_Demand_KW])

// Nombre total de mesures

[# Mesures] = COUNTROWS(Consommation)

// Écart entre pic et creux

[Amplitude] = [Conso max] - [Conso min]

9. Power BI : business intelligence

Mesures comparatives (Time Intelligence)

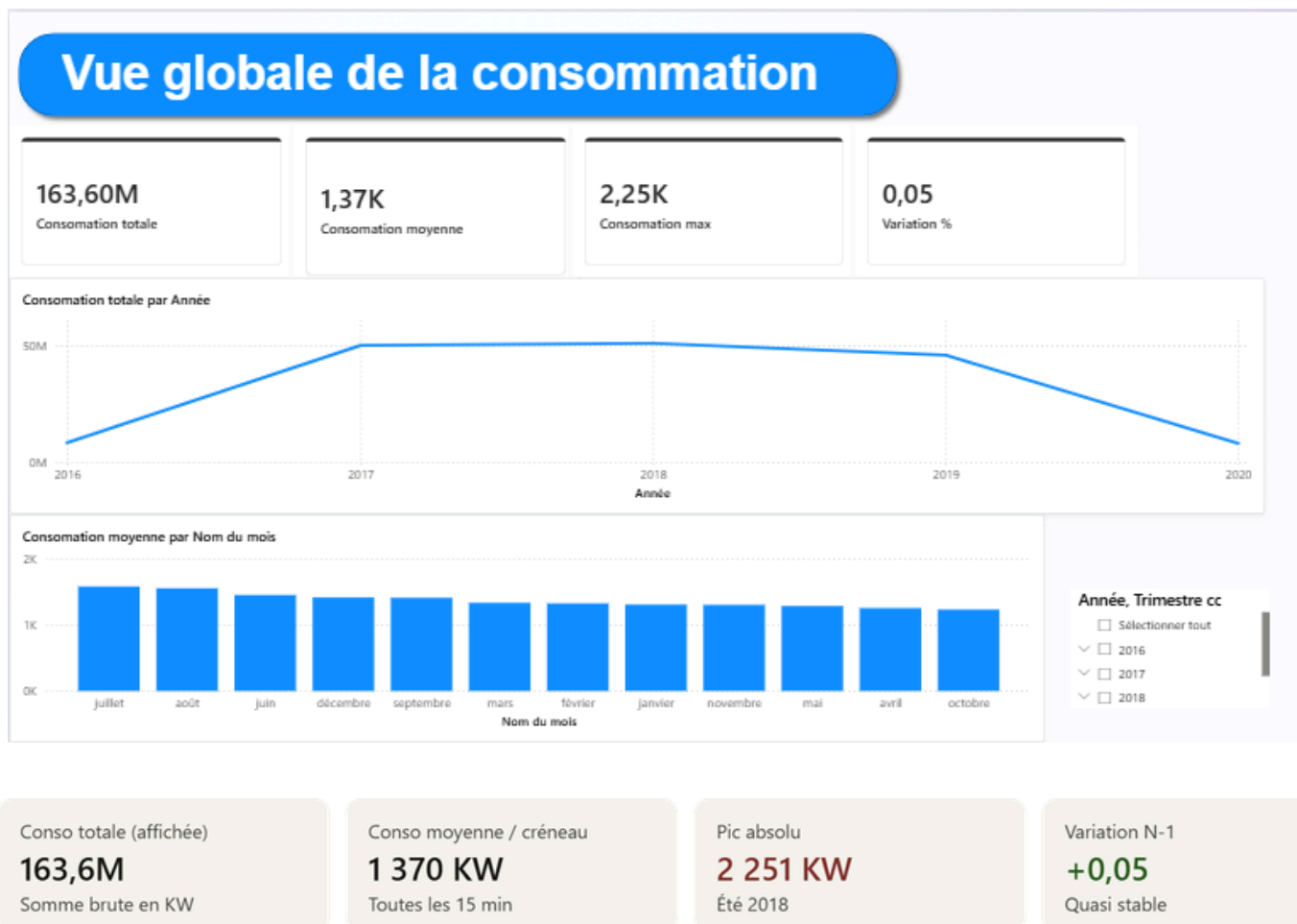
// Consommation même période année précédente

[Conso N-1] = CALCULATE([Conso totale], SAMEPERIODLASTYEAR(Dates[Date])) // Variation % vs année précédente [Variation % N-1] = DIVIDE([Conso totale] - [Conso N-1], [Conso N-1]) // Format : Pourcentage %

// Consommation week-end vs semaine [Conso Week-end] = CALCULATE([Conso moyenne], Consommation[Type de jour] = "Week-end") [Conso Semaine] = CALCULATE([Conso moyenne], Consommation[Type de jour] = "Jour ouvrable") // Ratio week-end / semaine [Ratio WE/Semaine] = DIVIDE([Conso Week-end], [Conso Semaine]) // Format : Pourcentage %

Les mesures Time Intelligence nécessitent que la relation avec la table Dates soit bien établie (étape 4).

9. Page 1 — Vue globale de la consommation



163,6M KW — un chiffre trompeur

C'est la somme de toutes les mesures de 15 min sur 3,5 ans. Ce nombre n'a pas de sens physique direct (on ne somme pas des puissances). Dans un vrai rapport, on parlerait plutôt de KWh consommés = Total_Demand_KW × 0,25h. Mentionne-le dans ta présentation pour montrer ta maîtrise du sujet.

1 370 KW en moyenne — le "rythme de croisière" du bâtiment

City Hall consomme en permanence ~1 370 KW, soit l'équivalent d'environ 1 370 ampoules de 1 kW allumées en continu. C'est la valeur de référence : tout ce qui dépasse est à expliquer (réunion, canicule, hiver rigoureux...).

9. Power BI : business intelligence

2 251 KW — le pic = alerte climatisation été 2018

Ce pic est +64% au-dessus de la moyenne. Juillet-août 2018 a connu une vague de chaleur à Boston. Le City Hall a maximisé la climatisation — c'est exactement le type d'anomalie qu'un système de prédiction doit anticiper pour éviter une surcharge réseau.

Variation +0,05 — consommation quasi stable sur la période

La variation annuelle est très faible. Cela signifie qu'il n'y a pas eu de changement majeur d'équipement ni de politique d'économie d'énergie significative entre 2016 et 2020. Bon contexte pour la prédiction : le comportement passé est représentatif du futur.

Graphique en courbes : Consommation par Date (depuis table Dates) → montre l'évolution Oct 2016 → Mars 2020

2016 et 2020 semblent faibles — mais c'est normal

Les données commencent en octobre 2016 et s'arrêtent en mars 2020. Ces deux années sont donc incomplètes. La courbe ne montre pas une baisse réelle de consommation mais un effet de coupure des données. 2018 est l'année complète la plus consommatrice (+2% vs 2017).

À préciser dans ta présentation : "2016 et 2020 sont des années partielles — les comparer aux autres serait trompeur."

Baisse notable en 2019 (−10% vs 2018)

La consommation passe de 51M à 45,9M entre 2018 et 2019. Plusieurs hypothèses possibles : hiver 2019 plus doux à Boston, mesures d'efficacité énergétique, ou réduction d'activité du bâtiment. C'est un signal important pour ton modèle de prédiction.

Histogramme : Consommation moyenne par Nom du mois → visualise la saisonnalité Boston

Été : juillet pic à 1 582 KWH
Hiver : déc-janv autour de 1 350–1 420 KW
Creux : oct-avr-mai les plus bas ~1 230–1 260 KW

Double pic été/hiver — signature d'un bâtiment de bureau

L'été monte à cause de la climatisation (juillet = +15% vs moyenne). L'hiver reste élevé à cause du chauffage. Avril, mai et octobre sont les mois les plus économes : température douce, pas de besoin de climatisation ni de chauffage intensif. Ce pattern saisonnier est le premier prédicteur de consommation — bien plus fiable que n'importe quel modèle ML seul.

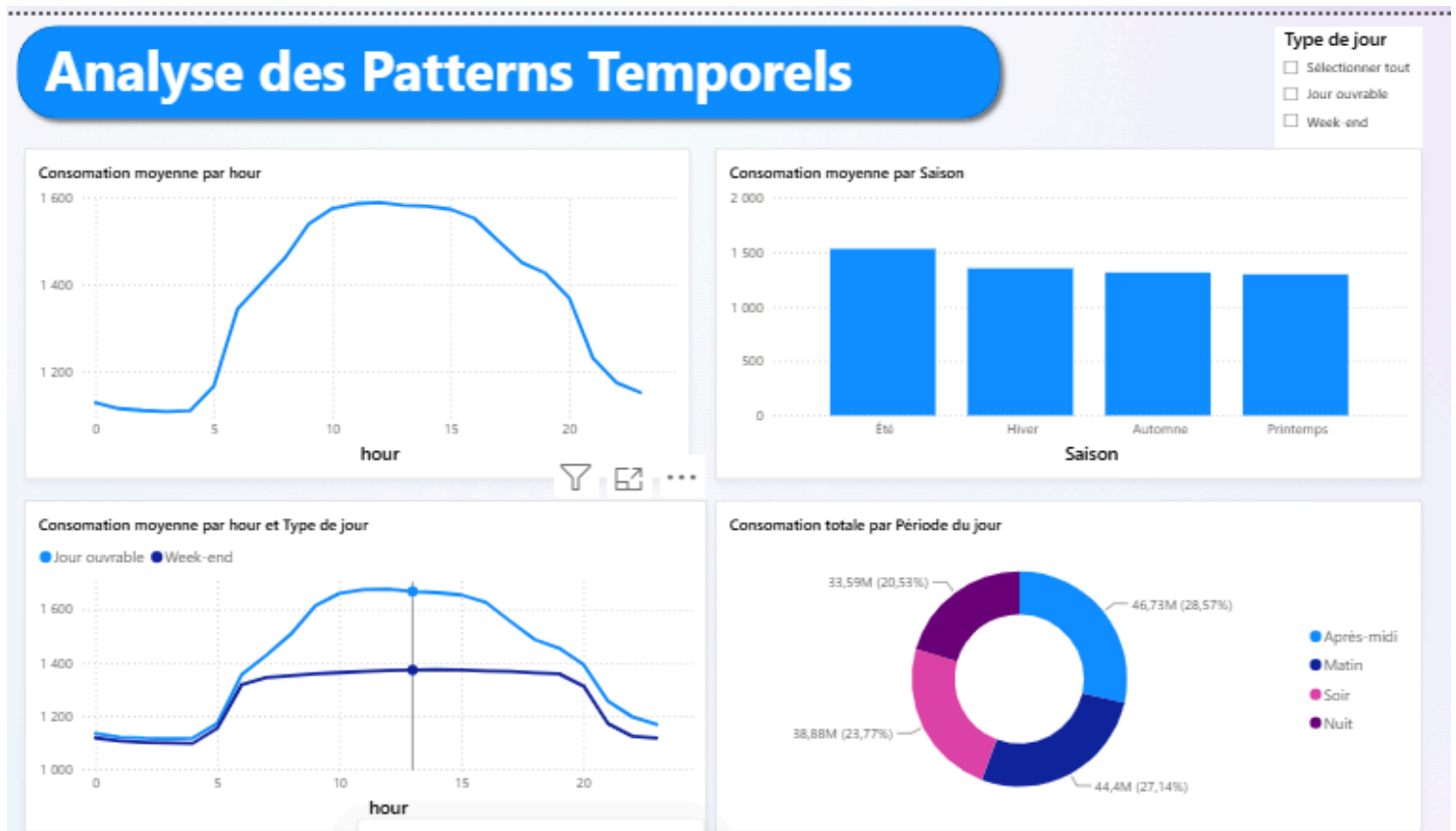
Cette page montre que la consommation du City Hall de Boston suit un cycle saisonnier clair, avec un pic estival lié à la climatisation et un maintien hivernal lié au chauffage. La stabilité de la variation annuelle (+5%) confirme que les données historiques sont fiables pour entraîner un modèle de prédiction.

Segments (filtres) : Année, Trimestre cc

10. Page 2 — Analyse des patterns temporels

- **Graphique en courbes : Conso moyenne par hour (0-23) → profil journalier typique du City Hall**
- **Graphique groupé : Même courbe Semaine vs Week-end → comparer les deux profils**
- **Graphique à barres : Conso moyenne par Saison → impact climatique de Boston**
- **Anneau : Répartition des mesures par Période (Matin/Après-midi/Soir/Nuit)**
- **Segment : Type de jour**

9. Power BI : business intelligence



Graphique en courbes : Conso moyenne par heure (0-23) → profil journalier typique du City Hall

La courbe en cloche est parfaite : la nuit le bâtiment est vide (1 109 KW minimum à 3h), puis dès 6h les systèmes s'allument automatiquement (ventilation, éclairage, serveurs). Le pic à 12h-13h correspond à la pleine activité. La descente progressive après 17h reflète la fin de journée de travail. C'est exactement le comportement attendu d'un hôtel de ville — et c'est une donnée très prévisible.

Graphique à barres : Conso moyenne par Saison → impact climatique de Boston

L'été domine nettement — Boston et la climatisation

L'été est +19% au-dessus du printemps. Boston a des étés chauds et humides (températures pouvant dépasser 35°C) → la climatisation du City Hall tourne à plein régime. L'hiver, bien que froid, est paradoxalement moins consommateur : les systèmes de chauffage modernes sont plus efficaces que la climatisation, et les bureaux génèrent leur propre chaleur.

Printemps = meilleure période pour l'efficacité énergétique

Mars-mai à Boston : température agréable, pas besoin de climatisation ni de chauffage intense. La consommation tombe à son minimum naturel (1 271 KW). C'est la consommation de base du bâtiment hors contraintes climatiques — une référence utile pour mesurer l'impact saisonnier.

Anneau : Répartition des mesures par Période (Matin/Après-midi/Soir/Nuit)

Après-midi + Matin = 55,7% de la consommation totale

Les heures de bureau (6h-18h) concentrent plus de la moitié de l'énergie consommée. C'est cohérent : personnel présent, équipements allumés, réunions, climatisation/chauffage à plein régime. La nuit représente seulement 20,5% mais ne descend jamais à zéro — preuve que le bâtiment consomme même sans occupants.

Résumé clé : heure + jour de la semaine + saison = les 3 variables qui prédisent 90% de la consommation. Le reste (~10%) est lié à des événements exceptionnels (canicule, réunion exceptionnelle, panne...).

9. Power BI : business intelligence

11. Page 3 — Prédiction et anomalies

- Graphique en courbes avec bande de prédiction : Ajoute une ligne de tendance via Analyse → Ligne de prévision (intégrée à Power BI)
- Nuage de points : lag_96 (axe X) vs Total_Demand_KW (axe Y) → montre la corrélation avec la veille
- Tableau matriciel : Conso par Année et Trimestre cc avec [Variation % N-1] en mise en forme conditionnelle
- Barres : Distribution par Niveau (Très basse → Très haute)



3 éléments visuels à distinguer

Ligne bleue continue données réelles 2016–2020

Ligne grise pointillée prédiction Power BI pour 2021

Zone grise large intervalle de confiance à 95%

L'intervalle de confiance signifie : "le modèle est sûr à 95% que la vraie valeur 2021 sera dans cette fourchette." Plus la fourchette est large, plus l'incertitude est grande — ce qui est normal après une perturbation comme le COVID.

Pourquoi la courbe chute si brutalement en 2020

Les données s'arrêtent en mars 2020 — le mois du premier confinement aux États-Unis. Le City Hall de Boston a fermé au public, réduisant la consommation de ~1 400 KW à ~1 000 KW (~28%). Cette chute est anormale par rapport aux 3 années précédentes et perturbe l'algorithme de prédiction qui "ne s'attendait pas" à une telle rupture.

Le biais COVID dans les données de fin de période explique pourquoi l'intervalle de prédiction 2021 est inhabituellement large. En conditions normales, la zone grise serait bien plus étroite.

9. Power BI : business intelligence

2017 : croissance généralisée sur tous les trimestres

Tous les trimestres 2017 sont verts. Le City Hall était en pleine activité, probablement avec plus d'événements ou de travaux que 2016. C'est l'année de référence maximale (+51% vs 2016, mais 2016 était partielle).

2018 : T3-T4 virent au rouge malgré un bon T1-T2

T1-T2 2018 sont encore positifs, mais T3 2018 (été) et T4 (fin d'année) montrent une baisse vs 2017. Pourtant 2018 T3 est l'été le plus chaud en absolu — mais légèrement sous le T3 2017. Signe que 2017 était une année exceptionnellement haute.

2019 : tous les trimestres en rouge — baisse structurelle

Le T3 2019 chute de -17% vs T3 2018 : l'été 2019 était plus frais à Boston. Le T4 2019 (-5%) confirme une tendance baissière. C'est probablement l'effet d'une meilleure efficacité énergétique ou d'un été climatiquement moins extrême.

2020 T1 : -29% — le signal COVID le plus visible du tableau

T1 2020 perd 29% vs T1 2019. C'est la signature directe du confinement de mars 2020. Le "Total 2020 = -0,82" est trompeur car il ne couvre que janvier-mars. À retenir : sans COVID, T1 2020 aurait été proche de 0% de variation.

Synthèse pour ta présentation : "Le tableau confirme que 2017-2018 sont les années de pic, 2019 amorce une baisse naturelle, et 2020 est perturbée par le COVID. La variation totale nette de +5% sur toute la période valide la stabilité du comportement énergétique du bâtiment."

10.Interface

L'application comporte quatre pages principales dans la barre latérale :

1. Dashboard

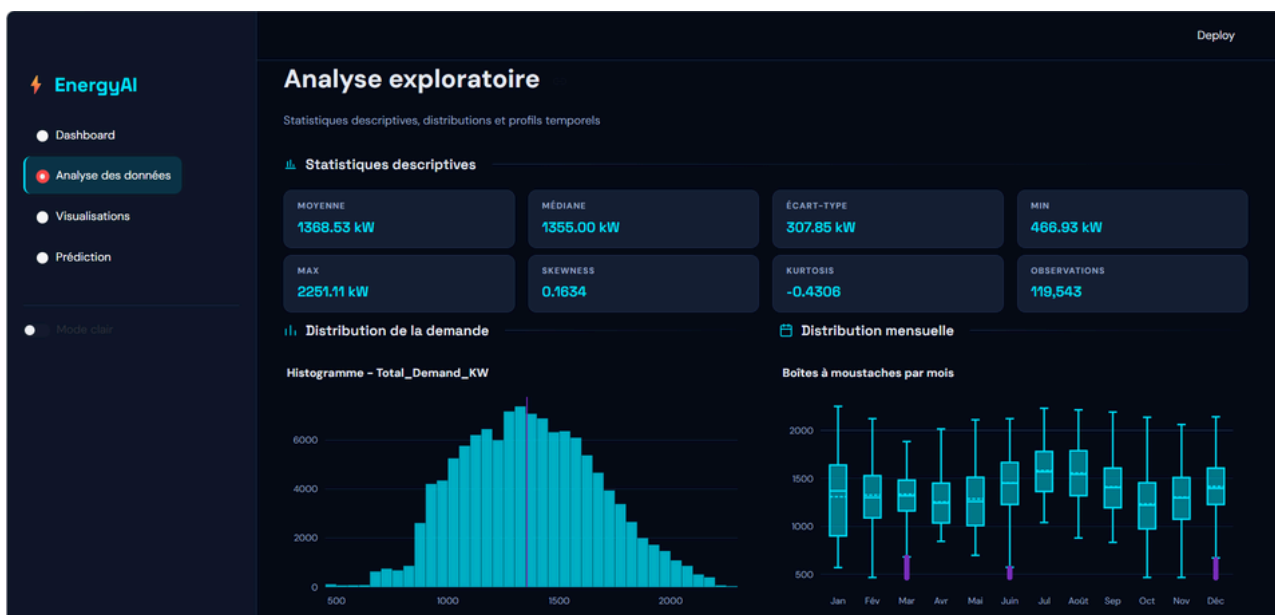
- Vue d'ensemble executive de la consommation électrique.
- Contenu :
 - Cartes avec :
nombre d'observations
consommation moyenne
pic de consommation
heure creuse
 - Série temporelle horaire de la demande
 - Profil mensuel moyen
 - Profil hebdomadaire moyen
 - Heatmap horaire / jour de semaine



10.Interface

2. Analyse des données

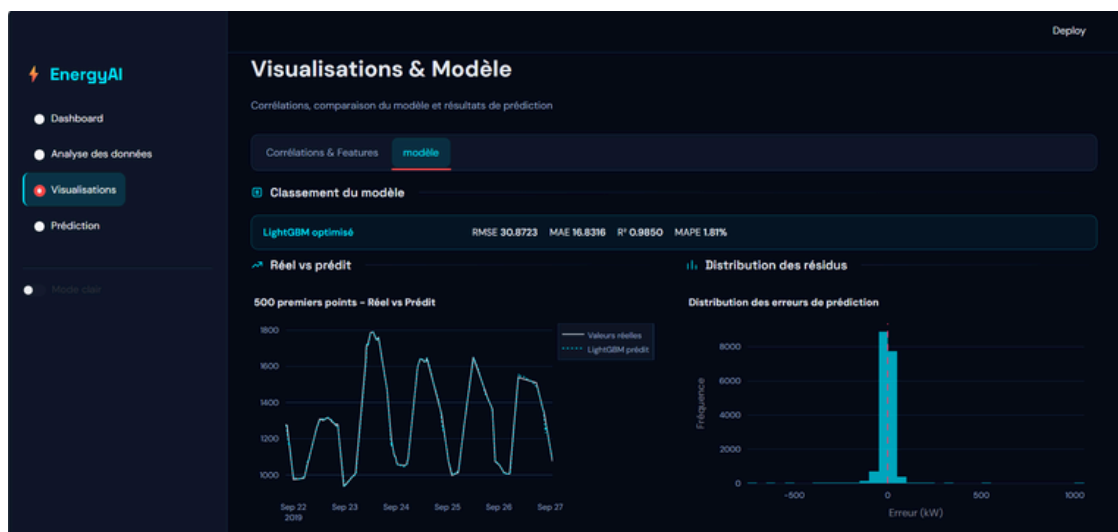
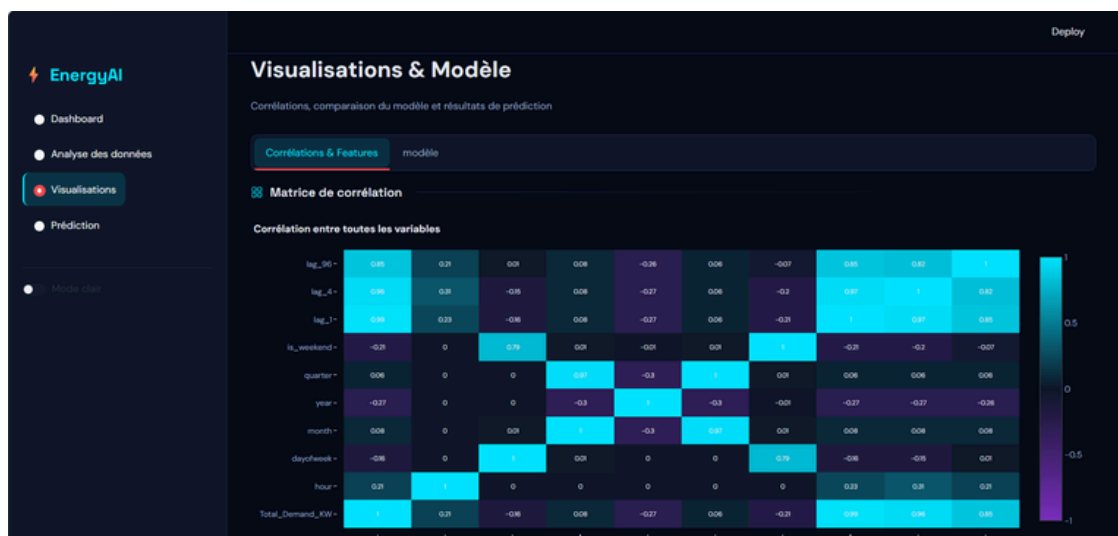
- Analyse exploratoire des données historiques.
- Contenu :
 - Statistiques descriptives : moyenne, médiane, écart-type, min, max, skewness, kurtosis, nombre d'observations
 - Histogramme de la distribution de la demande
 - Distribution mensuelle sous forme de boxplots
 - Analyse saisonnière par violon pour Printemps / Été / Automne / Hiver



10.Interface

3. Visualisations

- Formulaire de prédiction interactive.
- Contenu sous deux onglets:
 - Corrélations & Features
matrice de corrélation entre variables
corrélations de chaque variable avec la cible Total_Demand_KW
scatter plot "Demande actuelle vs Demande J-1 (lag_96)"
 - modèle
tableau de performance du modèle LightGBM optimisé
courbe réel vs prédit (premiers points)
distribution des résidus
scatter plot réel vs prédit



10.Interface

4. Prédiction

- Formulaire de prédiction interactive.
- Contenu sous:
 - Sélection de la date et de l'heure de prédiction
 - Choix du mois et du jour de la semaine
 - Saisie des valeurs de lag
lag_1 (15 min avant)
lag_4 (1 heure avant)
lag_96 (même heure la veille)
 - Bouton "Lancer la prédiction"
 - Résultat
carte de demande prédite
niveau de charge (faible / modérée / élevée)
gauge visuel
comparaison avec le profil horaire historique

